Занятие 8

Реализация онтологии и графа знаний

8.1. Цели и задачи работы

Цель работы: реализовать спроектированную на занятии 7 онтологию в графовой базе данных или специализированном инструменте, наполнить её данными из хранилища и создать граф знаний, пригодный для последующих запросов и анализа.

Задачи работы:

1. Выбрать инструмент реализации графа знаний и обосновать выбор.
2. Спроектировать структуру графа (типы узлов и типы рёбер) на основе онтологии.
3. Установить инструмент и настроить соединение.
4. Импортировать данные из реляционного хранилища (PostgreSQL) в граф.
5. Выполнить тестовые запросы к графу знаний.
6. Визуализировать фрагмент графа и сохранить изображение.

8.2. Теоретические сведения

8.2.1. Граф знаний и его отличие от реляционной БД

Граф знаний (Knowledge Graph) — способ представления знаний в виде помеченного графа, где узлы соответствуют сущностям (объектам предметной области), а рёбра — типизированным отношениям между ними. В отличие от реляционной базы данных, граф знаний не требует фиксированной схемы и позволяет легко добавлять новые типы связей без изменения структуры хранилища.

Ключевые отличия графа знаний от реляционной БД:

Аспект	Реляционная БД	Граф знаний
Единица хранения	Строка в таблице	Узел (node) или ребро (edge)
Связи	JOIN по внешним ключам	Ребро — первоклассная сущность с типом и атрибутами
Схема	Жёсткая (DDL)	Гибкая, schema-optional
Запросы	SQL	Cypher (Neo4j), SPARQL (RDF), Gremlin
Оптимальная задача	Агрегация, OLAP	Поиск путей, анализ связей, выводы

8.2.2. Инструменты для реализации графа знаний

Инструмент	Тип	Язык запросов	Сильные стороны	Ограничения
Neo4j	Графовая СУБД	Cypher	Визуализация, транзакции, ACID	Требует отдельного сервера

Apache Jena / TDB	RDF-хранилище	SPARQL	Поддержка OWL, стандарты W3C	Сложнее освоить
NetworkX (Python)	Библиотека в памяти	Python API	Нет сервера, простая установка	Нет персистентности
RDFLib (Python)	RDF-граф в памяти	SPARQL / Python API	Поддержка Turtle/RDF стандартов	Нет GUI

Для учебных целей рекомендуется Neo4j (удобный браузер) или NetworkX (без установки сервера). Выбор должен быть обоснован в отчёте.

8.2.3. Маппинг онтологии на граф

Онтология, спроектированная на занятии 7, переносится в граф по следующим правилам:

Элемент онтологии	Элемент графа	Пример (кинопрокат)
Класс (Class)	Метка узла (Label)	Film, Actor, Genre
Индивид (Individual)	Узел (Node)	(f:Film {title: "Матрица"})
Свойство данных (Data Property)	Атрибут узла	f.year = 1999
Объектное свойство (Object Property)	Ребро с типом	(actor)-[:ACTS_IN]->(film)

8.2.4. Основы языка Cypher (Neo4j)

Cypher — декларативный язык запросов Neo4j. Использует интуитивную нотацию «ASCII-арт» для обозначения графовых шаблонов:

Конструкция	Cypher-синтаксис	Смысл
Создание узла	CREATE (n:Film {title: "X"})	Узел с меткой Film и атрибутом title
Создание ребра	CREATE (a)-[:ACTS_IN]->(f)	Направленное ребро ACTS_IN
Поиск по шаблону	MATCH (a:Actor)-[:ACTS_IN]->(f:Film) WHERE f.year > 2000	Все актёры фильмов после 2000
Возврат результата	RETURN a.name, f.title LIMIT 10	Вывод полей с ограничением
Агрегация	RETURN a.name, count(f) AS films ORDER BY films DESC	Актёр и число его фильмов

8.3. Порядок выполнения работы

Шаг 1. Выбор и установка инструмента

Выберите один инструмент. Для Neo4j — запуск через Docker:

```
docker run --name neo4j -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/password -d neo4j:latest
```

Для NetworkX установка выполняется командой: pip install networkx matplotlib.
Обоснуйте выбор инструмента (доступность, функциональность, соответствие задаче).

Шаг 2. Проектирование структуры графа

На основе онтологии занятия 7 заполните таблицу маппинга:

Таблица 9.1 — Маппинг онтологии на структуру графа (заполнить):

Класс онтологии	Метка узла (Label)	Ключевые атрибуты узла	Уникальный идентификатор
…	…	…	…
…	…	…	…
…	…	…	…

Таблица 9.2 — Типы рёбер (заполнить):

Тип ребра	От метки	К метке	Атрибуты ребра	Кратность
…	…	…	…	…
…	…	…	…	…

Пример для кинопроката:

Тип ребра	От метки	К метке	Атрибуты ребра	Кратность
ACTS_IN	Actor	Film	role (String)	Многие ко многим
DIRECTED_BY	Film	Director	—	Многие к одному
BELONGS_TO	Film	Genre	—	Многие к одному
PRODUCED_BY	Film	Studio	—	Многие к одному

Шаг 3. Подключение к хранилищу и извлечение данных

Подключитесь к PostgreSQL и извлеките данные для каждого типа узлов. Для каждой таблицы из хранилища определите, какой тип узла она порождает и какие атрибуты используются. Пример SQL-запроса для кинопроката:

```
SELECT film_id, title, release_year, runtime_min FROM dim_film LIMIT 500
```

Шаг 4. Создание узлов графа

Напишите скрипт (Python или Cypher), создающий узлы каждого типа. Обеспечьте уникальность: используйте MERGE вместо CREATE, чтобы избежать дублирования. Пример для Neo4j:

```
MERGE (f:Film {film_id: $id}) SET f.title = $title, f.year = $year
```

Проверьте количество созданных узлов каждого типа. Зафиксируйте результаты в отчёте.

Шаг 5. Создание рёбер графа

На основе таблиц фактов и связей в хранилище создайте рёбра. Пример для связи ACTS_IN:

```
MATCH (a:Actor {actor_id: $aid}), (f:Film {film_id: $fid}) MERGE (a)-[:ACTS_IN {role: $role}]->(f)
```

Добавьте атрибуты рёбер там, где это предусмотрено онтологией (вес, дата, роль).

Шаг 6. Тестовые запросы к графу

Выполните минимум 4 запроса, проверяющих корректность построенного графа:

№	Описание запроса	Ожидаемый результат
1	Найти все связи заданного узла	Список соседних узлов с типами рёбер
2	Найти кратчайший путь между двумя узлами	Последовательность узлов и рёбер
3	Найти 10 узлов с наибольшим числом связей	Топ-10 по степени
4	Выполнить запрос, характерный для вашей задачи	Результат, значимый для предметной области

Шаг 7. Визуализация фрагмента графа

Постройте визуализацию подграфа из 15–30 узлов. Для Neo4j используйте Neo4j Browser (команда MATCH (n) RETURN n LIMIT 25). Для NetworkX используйте matplotlib. Сохраните изображение в файл PNG и приложите к отчёту. Цветовое кодирование: разные типы узлов — разными цветами.

8.4. Структура отчёта

1. Титульный лист.
2. Цель работы и задачи.
3. Выбор инструмента и обоснование.
4. Таблица маппинга онтологии на структуру графа (таблицы 9.1 и 9.2 — заполненные).
5. Описание процесса импорта данных (скрипты, количество узлов и рёбер по типам).
6. Тестовые запросы с текстом запроса и результатами (не менее 4 запросов).
7. Визуализация фрагмента графа (скриншот или сохранённый файл).
8. Выводы: что удалось реализовать, какие трудности возникли.

8.5. Контрольные вопросы

1. Чем граф знаний отличается от реляционной базы данных? Когда предпочтительнее использовать граф?
2. Что такое узел (node) и ребро (edge) в контексте графовой базы данных? Какие атрибуты могут иметь рёбра?
3. Объясните принцип работы операции MERGE в Cypher. Почему она предпочтительнее CREATE при импорте данных?
4. Что такое кратчайший путь в графе? Какой алгоритм используется для его поиска в Neo4j?
5. Каким образом онтология OWL соотносится с графом в формате RDF? В чём разница между RDF и Neo4j?

8.6. Литература

1. Добров, Б. В. Онтологии и тезаурусы (см. полное описание выше) — практическая часть про построение и наполнение онтологий.

2. Гаврилова, Т. А. Базы знаний интеллектуальных систем (см. полное описание выше) — глава о формальных моделях онтологий и их машинном представлении.
3. Осипов, Г. С. Методы искусственного интеллекта (см. полное описание выше) — раздел о гетерогенных семантических сетях как основе графа знаний.
4. Люгер, Дж. Ф. Искусственный интеллект (см. полное описание выше) — глава о представлении знаний фреймами и сетями, применимая к реализации графа.
5. Рассел, С. Искусственный интеллект: современный подход (см. полное описание выше) — глава Knowledge Representation с формальным аппаратом для реализации онтологий.